



UNIVERSIDAD NACIONAL AUTÓNOMA DE  
MÉXICO

FACULTAD DE INGENIERÍA  
DIVISIÓN DE INGENIERÍA ELÉCTRICA  
INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e  
INTERACCIÓN HUMANO COMPUTADORA



## **EJERCICIOS DE CLASE N° 01**

**NOMBRE COMPLETO: Cordero Miranda Evelyn Patricia**

**N° de Cuenta: 317314975**

**GRUPO DE LABORATORIO: 13**

**GRUPO DE TEORÍA: 06**

**SEMESTRE 2026-2**

**FECHA DE ENTREGA LÍMITE: 14 de febrero de 2026**

**CALIFICACIÓN: \_\_\_\_\_**

## Actividades realizadas

### 1.- Cambio de color

Para esta primera actividad se pidió realizar el cambio de color de fondo de pantalla entre rojo, verde y azul de forma cíclica y solamente mostrando los tres colores mencionados en un periodo de lapso adecuado para el ojo humano, eligiendo finalmente un cambio de cada 1.5 segundos.

Se comenzó haciendo un control para el tiempo, agregando variables que nos ayudarían a tener un mejor control, además de agregar un arreglo que contiene los colores a mostrar y un index para su manejo eficiente.

```
192 // Controladores de tiempo para cambiar el color cada 1.5 segundos
193 float tiempoAnterior = 0.0f;
194 int indiceColor = 0;
195 float colores[3][3] = {
196     {1.0f, 0.0f, 0.0f}, // Rojo
197     {0.0f, 1.0f, 0.0f}, // Verde
198     {0.0f, 0.0f, 1.0f}  // Azul
199 };
```

Dentro del while que se ejecuta cada ciclo de reloj mientras la ventana emergente no se cierre (`while (!glfwWindowShouldClose(mainWindow))`) se agregó el controlador de cambios de color. El tiempo actual se calculó mediante la función `glfwGetTime()` que nos devuelve el tiempo transcurrido en segundos, después verificamos con un condicional if que, si este tiempo tomado menos el tiempo anterior es mayor o igual a los 1.5 segundos que necesitábamos, actualiza nuestro index que se usa para mostrar las distintas paletas de colores y actualiza el tiempo anterior al tiempo actual.

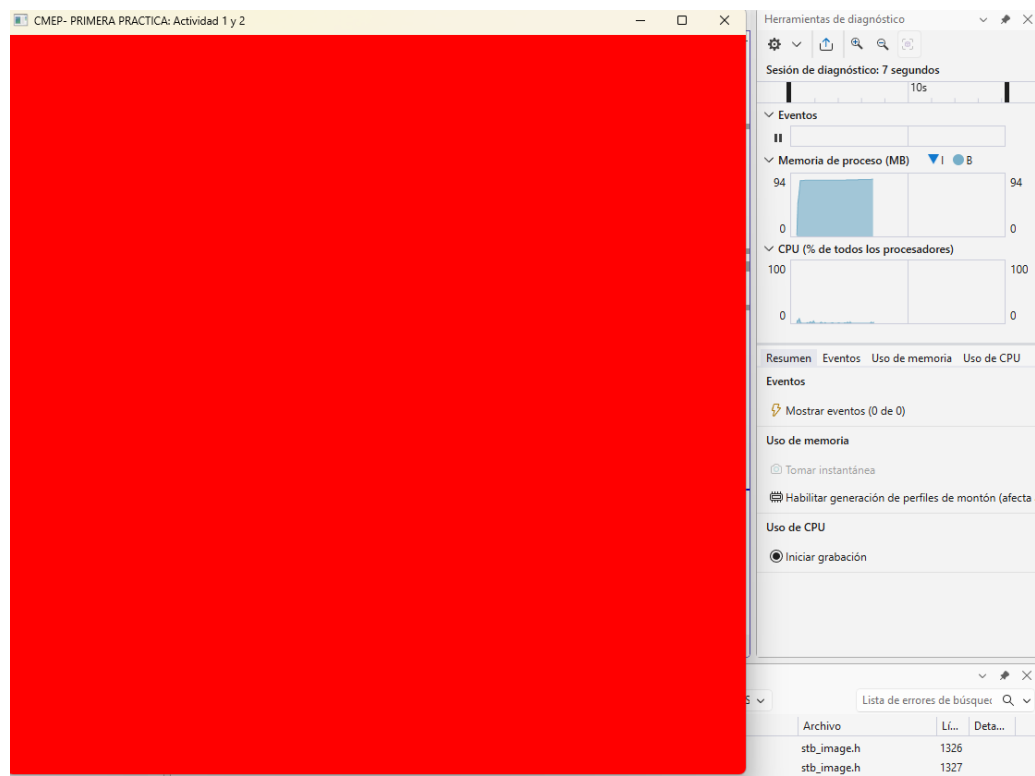
Finalmente, con la instrucción `glClearColor()` cambiamos un poco la forma en que le damos los parámetros usando el arreglo de colores y el index que se mencionó.

```
207 float tiempoActual = (float)glfwGetTime();
208 if (tiempoActual - tiempoAnterior >= 1.5f) {
209     indiceColor = (indiceColor + 1) % 3;
210     tiempoAnterior = tiempoActual;
211 }
212
213 //Limpiar la ventana
214 glClearColor(colores[indiceColor][0], colores[indiceColor][1], colores[indiceColor][2], 1.0f);
```

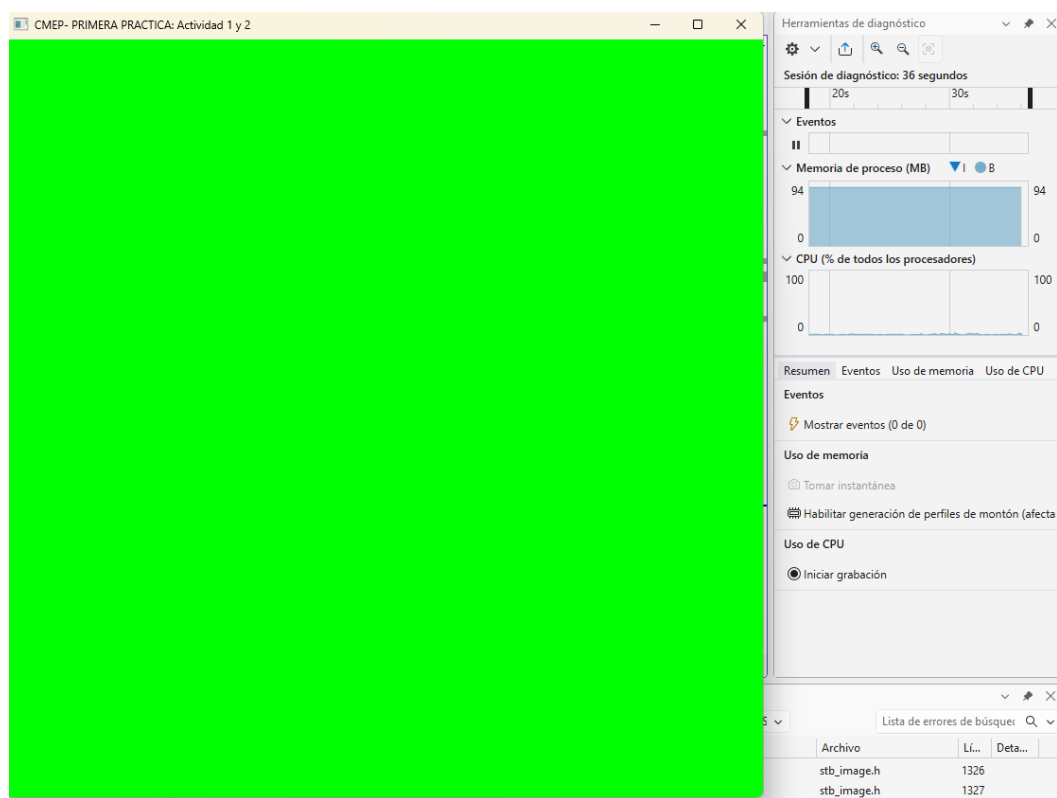
### EJECUCIÓN:

Se adjunta la ventana “Herramientas de diagnóstico” en la captura para que se pueda visualizar los segundos transcurridos y que se verifique así el cambio de color.

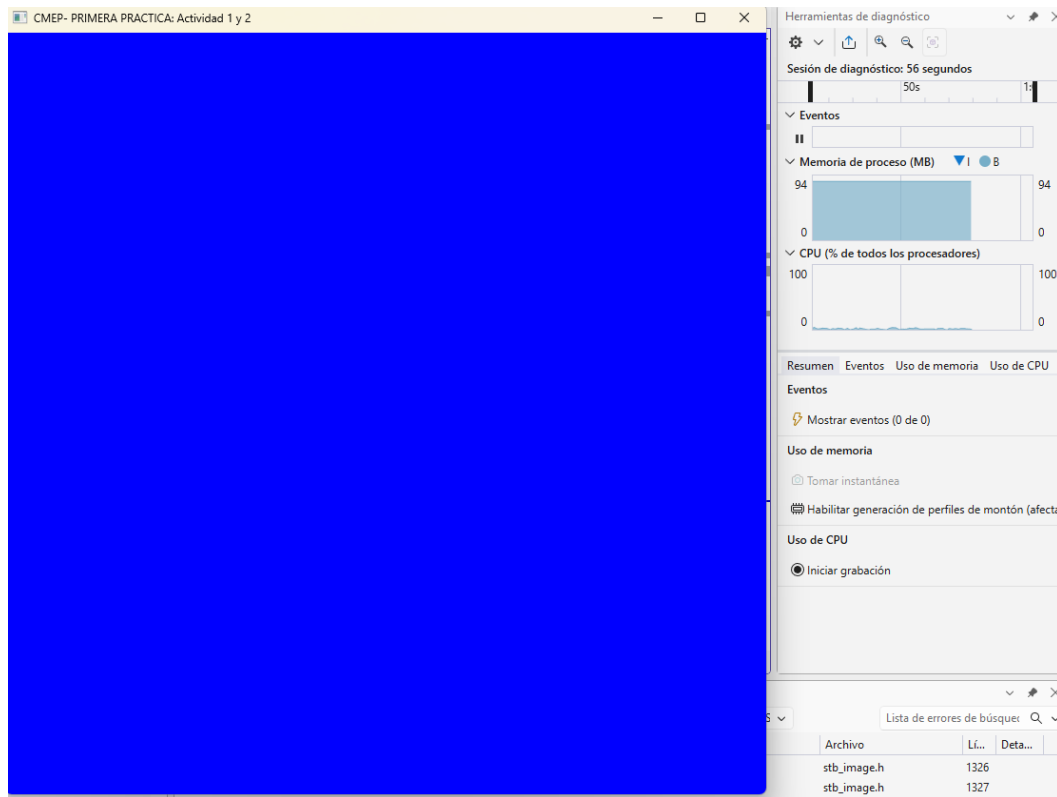
Color ROJO



Color VERDE



## Color AZUL



## 2.- Rombo y trapecio

Para la segunda actividad se pidió dibujar en pantalla de manera simultánea en ventana un rombo y un trapecio isósceles, con un total de 6 triángulos

Ya que se siguió usando el main principal donde estaba el cambio de color cada 1.5 segundos, se decidió dejar ese cambio activo y cambiar el color de las figuras que mostraremos a un color blanco en lugar de rojo.

```
19 // Fragment Shader con color blanco para las figuras
20 //recibir Vcolor y dar de salida color
21 static const char* fShader = "
22 #version 330
23 out vec4 color;
24 void main()
25 {
26     color = vec4(1.0f, 1.0f, 1.0f, 1.0f);
27 }";
```

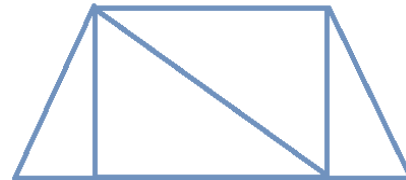
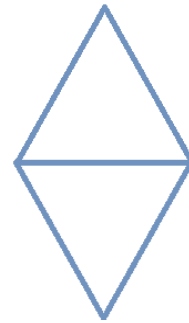
El segundo cambio más importante fue dentro de la función `CrearTriangulo()`, ya que pasamos de un solo arreglo de 3 vértices a uno de 18, esto se debe a que, sabiendo que

ocuparíamos un total de 6 triángulos para realizar las figuras, inferimos entonces que requeríamos de 3 vértices por cada uno.

```

29 void CrearTriangulo()
30 {
31     GLfloat vertices[] = {
32         // Como estamos en 2D el ultimo valor (Z)
33         // lo dejamos siempre en 0 ---- X,Y,Z
34
35         //ROMBO: Conformado por dos triángulos
36         // Triángulo de arriba
37         -0.8f, 0.0f, 0.0f, // Izq
38         -0.5f, 0.45f, 0.0f, // Arriba
39         -0.2f, 0.0f, 0.0f, // Der
40         // Triángulo de abajo
41         -0.8f, 0.0f, 0.0f, // Izq
42         -0.5f, -0.45f, 0.0f, // Abajo
43         -0.2f, 0.0f, 0.0f, // Der
44
45         //TRAPECIO: Conformado por 4 triángulos
46         // (2 para el rectángulo central
47         // y 1 para cada triángulo lateral
48         // Rectángulo central (2 triángulos)
49         //Izquierdo
50         0.0f, -0.2f, 0.0f,
51         0.4f, -0.2f, 0.0f,
52         0.0f, 0.3f, 0.0f,
53         //Derecho
54         0.0f, 0.3f, 0.0f,
55         0.4f, -0.2f, 0.0f,
56         0.4f, 0.3f, 0.0f,
57         // Triángulo izquierdo
58         -0.2f, -0.2f, 0.0f,
59         0.0f, -0.2f, 0.0f,
60         0.0f, 0.3f, 0.0f,
61         // Triángulo derecho
62         0.4f, -0.2f, 0.0f,
63         0.6f, -0.2f, 0.0f,
64         0.4f, 0.3f, 0.0f
65     };

```



Finalmente, cambiamos los parámetros de para recibir los 18 vértices totales.

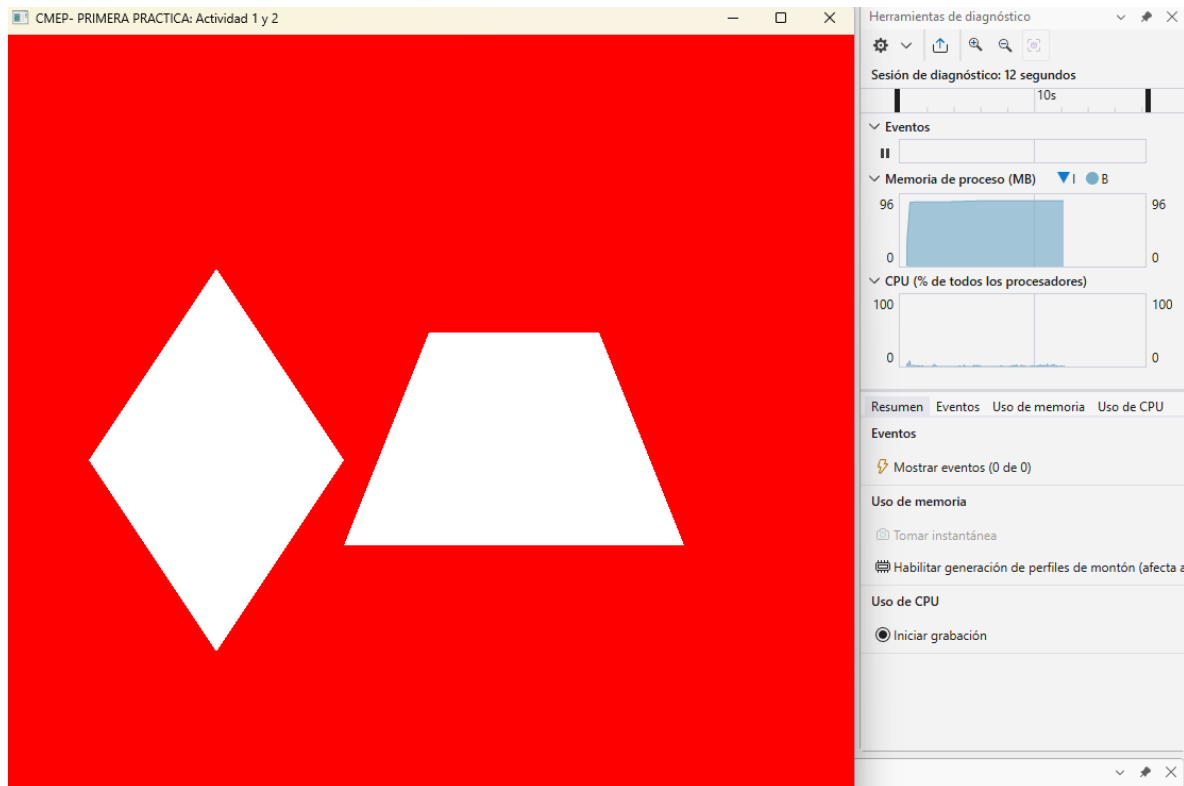
```

215 glUseProgram(shader);
216 glBindVertexArray(VAO);
217
218 // DIBUJAR: Ahora son 18 vértices en total (6 del rombo + 12 del trapecio)
219 glDrawArrays(GL_TRIANGLES, 0, 18);

```

## EJECUCIÓN:

Considerar que el cambio de color sigue presente.



## Problemas presentados

Al no recordar mucho de C++ se tuvo que repasar la documentación básica en cuanto a sintaxis refiere y aunque esto fue lo que causó mayor problema en la ejecución quizá el verdadero problema recae en el uso de las coordenadas de los vértices pues en más de una ocasión se pensaba que las coordenadas propuestas eran las adecuadas y al ejecutar el programa las figuras que se mostraban no eran deseadas. Al intentar arreglar esta situación y proponer otras coordenadas el problema persistía, dicho de otra manera, las figuras salieron de la forma más cercana posible a lo esperado mediante prueba y error.

## Conclusión

Sobre los ejercicios de clase:

Creo que su nivel de dificultad fue bajo, como una primera actividad de laboratorio fueron adecuados y como el primer acercamiento al código fueron lo ideal, algunas funciones fueron en inicio complejas de interpretar, pero con un repaso continuo no hay duda que se podrán manejar de manera más natural. Se logró comprender de manera media el uso de los vértices y su repercusión en las coordenadas, aunque quizá se necesite más práctica para su mejor uso ya que la forma de colocarlos no es muy intuitiva.

### Comentarios generales

Quizá la explicación de la actividad pudo tener un poco más de detalles, ya que no se logro acabar en el laboratorio verificar las instrucciones en la plataforma dejaron preguntas al aire, aunque se comprende que esto es más problema de uno como alumno que de la actividad pedida por el profesor.